

Design_Rationale_REQ3

A. NPC Implementation

Design 1: Create NPC-specific classes that inherit directly from Actor.

Pros	Cons
Each NPC can implement logic specific to its needs without constraints from a base class.	Common logic must be re-implemented in each NPC class.
NPCs with minimal or no monologue logic can avoid inheriting unnecessary functionality.	Adding new shared features would require modifying every NPC class instead of a single base class.
The class hierarchy is flattened, reducing structural complexity.	The lack of a shared abstraction for conversational NPCs makes the system less modular and harder to extend.

Design 2: Create an abstract class with shared logic for NPCs, which the NPCs inherit from.

Pros	Cons
The abstract ConversationalActor class makes it easy to create new NPCs by reusing shared features.	Similar NPC behaviors need to be updated individually, which is time-consuming and error-prone.
Monologues can be added with or without conditions, supporting context-sensitive dialogue.	New NPCs add only the behaviors they need, but shared behaviors may lead to duplicated code, increasing maintenance effort.
Changes to the listening logic only need to be made in one place, simplifying debugging and updates.	If some NPCs need specialized listening logic, the base class may become too generic to meet their needs.
The abstract initMonologues() method provides a clear extension point for adding NPC-specific dialogue.	
Each NPC class focuses on its monologues and behaviors, while the base class manages the mechanics.	

B. Monologue Implementation

Design 1: Composite Condition Pattern

Pros	Cons
Supports diverse checks (money, inventory, attributes) via the Condition interface.	More classes/interfaces to maintain, increasing complexity.
Add new condition types without changing core logic.	

Design 2: Embed monologue in conversational actor class

Pros	Cons
All monologue-related logic is centralized within the ConversationalActor, reducing the need for additional classes.	Monologue logic is tightly coupled with the actor, making it harder to reuse or extend for other types of actors.
Reduces the number of classes and interactions, simplifying the overall system.	If other actors need monologues, the logic must be duplicated or refactored.
	As monologues and conditions increase, the ConversationalActor class may become bloated and harder to maintain.

C. Condition Implementation

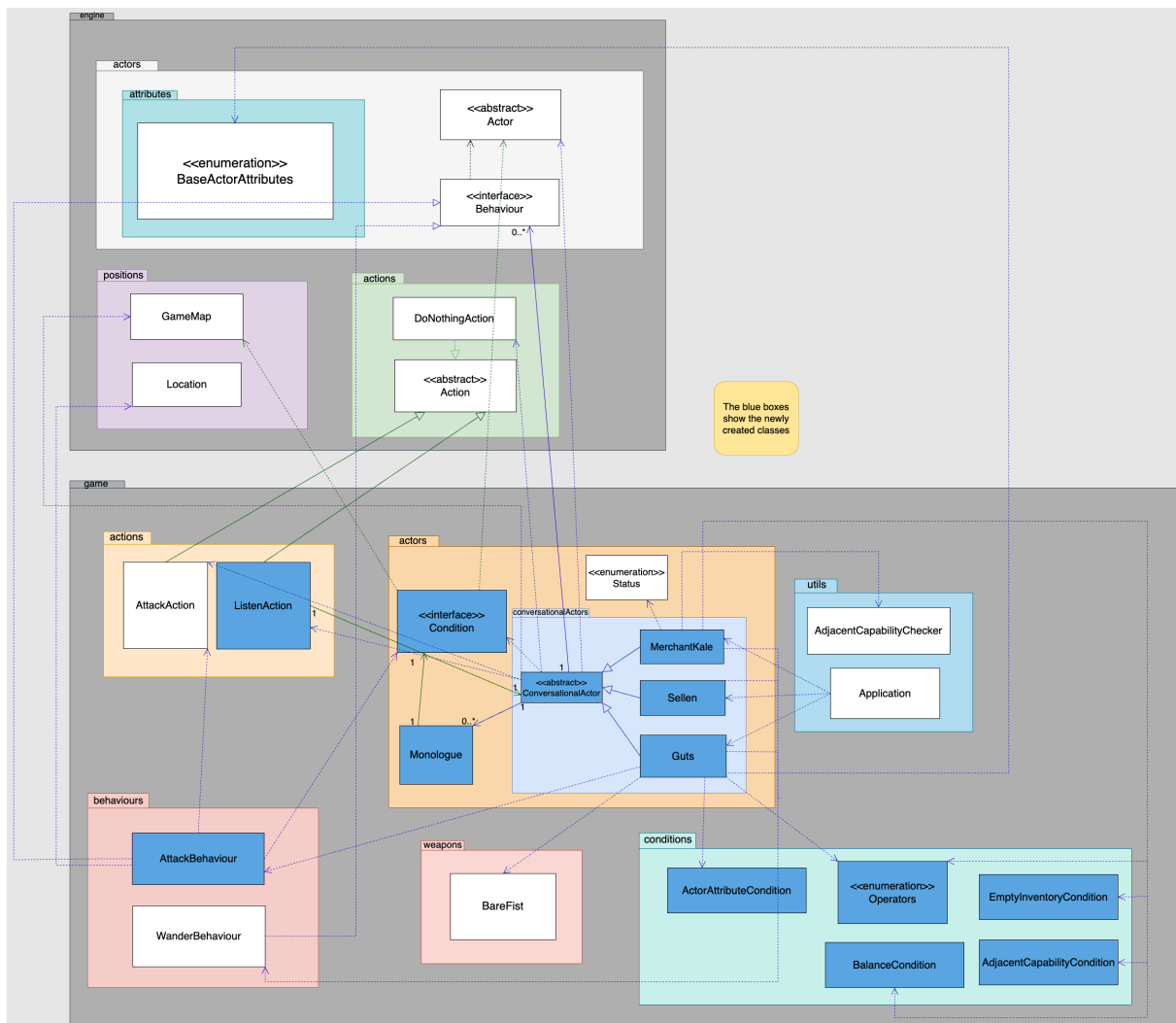
Design 1: Generic Condition Class

Pros	Cons
Single class handling all conditions can be easier to read for small projects.	Harder to see what each instance does at a glance.
No need to create a new class for each new condition.	One class may get messy with lots of if/else or switch statements.
	As conditions grow, the class may become tangled and error-prone.

Design 2: Dedicated Condition Classes

Pros	Cons
Each class has a clear purpose and is easy to understand	Adding a new condition requires creating a new class

Pros	Cons
Can provide specialized logic for specific conditions	Difficult to manage all condition classes uniformly
Error isolation; problems in one class don't affect others	
Reusable in different contexts	



The diagram above shows the final design of requirement 3, which utilizes Design 2 for npc and condition implementation, Design 1 for monologue implementation.

Each class adheres strictly to the **Single Responsibility Principle (SRP)**, with ConversationalActor handling only dialogue flow and monologue selection, Monologue encapsulating its text and the trigger Condition, ListenAction

displaying speech, NPC subclasses like Sellen, MerchantKale and Guts defining only their own names, stats, behaviors and monologues, and AttackBehaviour scanning nearby locations for enemies and when one is found creating an AttackAction using the actor's weapon.

The Condition interface and its integration with the Monologue class create a flexible, maintainable, and extensible dialogue system that follows the **Open-Closed Principle (OCP)**. With a standardized interface, new condition types can be added without changing existing code. Monologue evaluates conditions through this interface, keeping logic decoupled and easy to extend. Conditions are reusable, composable, and can be assigned dynamically. NPCs can define custom dialogues in `initMonologues()` without modifying the base `ConversationalActor`, keeping the system stable as it evolves.

All subclasses adhere to the **Liskov Substitution Principle (LSP)**, so any `ConversationalActor` reference in client code seamlessly accepts instances of Sellen, MerchantKale, or Guts. Each override of `initMonologues()` and every custom behavior respects the contracts defined in the abstract base, ensuring no unexpected side-effects or broken interactions when substituting one NPC type for another.

The **Interface Segregation Principle (ISP)** is reflected in narrowly scoped interfaces. The Condition interface provides only what it needs, avoiding "fat" interfaces that would force classes to implement unused methods. This lightweight interface design reduces coupling and enhances clarity.

The dialogue system design adheres to the **Dependency Inversion Principle (DIP)** by having high-level modules depend on the Condition abstraction rather than concrete implementations. Both Monologue and ConversationalActor interact solely with this interface, decoupling core dialogue logic from specific conditions. Concrete condition classes implement the interface, enabling independent evolution without impacting higher-level components. This separation fosters flexible condition composition, reuse across NPCs, and easy integration of new behaviors.

Duplication is minimized via the **Don't Repeat Yourself (DRY)** principle by placing shared logic in `ConversationalActor` and NPC-specific data in subclasses. The design balances inheritance for NPC hierarchy with composition (e.g., maps/lists) for managing monologues and behaviors. Following **Composition Over Inheritance**, new monologue-triggering logic can be added with minimal disruption. Conditions enable flexible, context-aware dialogue without cluttering classes.

From a **connascence** perspective, our design enhances maintainability by managing key coupling types. It ensures consistent method names (**Connascence of Name**) via the Condition interface and BalanceCondition class. Interfaces enforce method signatures and return types to handle **Connascence of Algorithm and Type**. The Operators enum replaces primitive values to reduce Connascence of Value, while constructor parameters enable logic changes without code edits, minimizing **Connascence of Timing and Value**.

Using the Condition interface to add new dialogue conditions via new classes, avoiding changes to existing code. CompositeCondition enables combining triggers flexibly. NPCs are extended by subclassing ConversationalActor and mixing in behaviors without altering others. New states like mood integrate through the Actor attribute system and ActorAttributeCondition. Inter-NPC interactions build on extendable AdjacentCapabilityCondition for dynamic cross-talk.

By combining inheritance for NPC hierarchy and composition for behaviors and conditions, the architecture remains modular and maintainable, open for extension but closed to modifying core logic, allowing evolution without losing stability or clarity.